pg_project 운영(관리) 가이드

```
대상      : naturalTuna/pg_project 로 설치한 PostgreSQL (single / HA)
작성일    : 2026-11-09
같이 보기 : 아키텍처_가이드.pdf (그림으로 보는 구성 / 장애 동작)
            설치가이드.txt (설치, 테스트, 재설치)
            pgmon_가이드.txt (모니터링 상세)

경로는 기본값 기준.  다르면 바꿔서 읽을 것
  엔진 /postgres/app    데이터 /pgdata/data    로그 /pgdata/log
  아카이브 /pgbackup/arc    백업 /pgbackup/back    pgBackRest 저장소 /pgbackup/pgbackrest
  patroni.yml = /postgres/app/etc/patroni/patroni.yml  (아래에서 $PYML)
```

목차

1. 일상 점검 체크리스트
2. 기본 명령 (환경 / 기동 / 정지)
3. Patroni 운영 (HA)
4. VIP 운영 (HA)
5. 안전장치 / 노드 추가 / DR (HA)
6. 백업 / WAL 아카이브 / 복구
7. pgmon 모니터링
8. 쿼리 튜닝 - pg_tune
9. 장애 대응 플레이북
10. 로그 위치 모음
11. 정기 작업 / 변경 작업

1. 일상 점검 체크리스트

```
[매일]
 [ ] patronictl -c $PYML list            Leader 1, Replica streaming, Lag 작음   (HA)
 [ ] bash ~/pgvip/vip_watch.sh --once    OK                        (HA, etcd 서버)
 [ ] tail /pgdata/log/pg_basebackup.log  어젯밤 백업 성공
     (pgbackrest) pgbackrest --stanza=<stanza> info   마지막 백업 시각 / status ok
 [ ] psql -c "select failed_count, last_failed_time from pg_stat_archiver;"
 [ ] df -h /pgdata /pgbackup             80% 미만
 [ ] pgmon 대시보드 / Alert 화면          활성 Alert 없음

[매주]
 [ ] pgmon Report: growth (테이블 증가), repl (복제 지연 추이)
 [ ] pgmon Vacuum: XID wraparound 위험 테이블
 [ ] pgmon Object: 미사용 인덱스, bloat
 [ ] /pgdata/log/wal_cleanup.log         정리 정상
 [ ] pitr_restore.sh --list              백업 개수 = 보관 개수
     (pgbackrest) pgbackrest --stanza=<stanza> info   full 개수 = 보관 개수
 [ ] bash pgmon_check.sh                 pgmon 수집 정상

[분기]
 [ ] switchover 훈련 (3-3)
 [ ] 복구 리허설: 별도 서버에서 pitr_restore.sh / pgbr_restore.sh 로 복원 확인
 [ ] bash ha_test.sh  (업무 영향 없는 시간)
 [ ] 패스워드 / pg_hba 점검
```

2. 기본 명령 (환경 / 기동 / 정지)

```
~/.postgresrc 가 .bashrc 에서 로드됨 (PGHOME, PGDATA, PGPORT, PATH)

alias
  pgstart  pgstop  pgstat  pgreload     pg_ctl 래퍼  (single 전용)
  pgbin pgetc pgdata pgback pgdump pglog    디렉토리 이동

[single]
```

```
pgstart / pgstop / pgstat
pgreload                            postgresql.conf 리로드
psql -c "select pg_reload_conf();"    (같은 효과)

[HA]  ※ pg_ctl 로 직접 기동/정지하지 말 것. Patroni 가 다시 띄우거나 failover 함
  sudo systemctl status patroni
  sudo systemctl stop patroni       그 노드 PG 도 정지 (Primary 면 failover 발생)
  sudo systemctl start patroni
  sudo systemctl status etcd

[노드별 Patroni 정지 시 동작]
  Primary 정지 : PG 도 같이 내려감 -> ttl(30초) 후 Standby 자동 승격
                 다시 start 하면 Replica 로 복귀 (pg_rewind)
  Standby 정지 : 복제만 끊김, Primary 는 계속. start 하면 자동 재합류
  ※ pg_ctl 로 직접 PG 를 내리면 Patroni 가 다시 띄우거나 충돌. 항상 Patroni 로 제어
```

2-1. sudo 권한 축소 (운영 전 선택)

```
설치 때는 NOPASSWD: ALL 이 필요하지만 운영 중에는 아래 정도면 된다
  postgres ALL=(root) NOPASSWD: /usr/bin/systemctl start patroni, \
      /usr/bin/systemctl stop patroni, /usr/bin/systemctl restart patroni, \
      /usr/bin/systemctl status patroni, /usr/sbin/ip, /usr/sbin/arping
patronictl 은 sudo 불필요. 노드 추가 / 재설치 때는 다시 넓혀야 함
```

3. Patroni 운영 (HA)

```
PYML=/postgres/app/etc/patroni/patroni.yml
CL=<클러스터명>    (기본 pg-cluster)
```

3-1. 상태

```
patronictl -c $PYML list
patronictl -c $PYML history           failover / switchover 이력
patronictl -c $PYML show-config       DCS 설정 (ttl, failsafe, sync ...)
curl -s http://<노드>:8008/patroni | python3 -m json.tool
curl -s -o /dev/null -w '%{http_code}\n' http://<노드>:8008/primary   200=리더
```

3-2. 파라미터 변경

```
patronictl -c $PYML edit-config       DCS 의 postgresql.parameters 수정
patronictl -c $PYML list              "Pending restart" 표시 확인
patronictl -c $PYML restart $CL       재시작 필요 항목 반영 (노드 순차)
patronictl -c $PYML reload $CL        재시작 불필요 항목 반영
                                      (각 노드 patroni.yml 도 다시 읽음)

※ postgresql.conf 직접 수정은 Patroni 가 덮어쓸 수 있음. edit-config 를 쓸 것
```

3-3. switchover (계획 전환, 무손실)

```
patronictl -c $PYML switchover $CL
  -> 후보 / 예약 시각 물어봄. 앱은 VIP 로 재접속
점검 작업(OS 패치 등)은 Replica 먼저 작업 -> switchover -> 나머지 작업
```

3-4. 수동 failover (Leader 가 이미 비정상일 때)

```
patronictl -c $PYML failover $CL
```

3-5. 자동 failover 일시 중지 (작업 중 오작동 방지)

```
patronictl -c $PYML pause
... 작업 ...
patronictl -c $PYML resume
※ pause 동안에는 Primary 가 죽어도 failover 안 함. 작업 후 반드시 resume
```

3-6. Replica 재구성 (복제 깨짐, 디스크 교체 등)

```
patronictl -c $PYML reinit $CL <replica 이름>
  -> Leader 에서 pg_basebackup 으로 다시 클론
```

3-7. 노드 이름 / 역할 확인

```
patroni.yml 의 name: 값이 patronictl list 의 Member
```

4. VIP 운영 (HA)

```
구성
  각 DB 노드  /postgres/app/etc/patroni/vip_callback.sh  Patroni callback (즉시)
              /postgres/app/etc/patroni/vip_check.sh     crontab 매분, 20초 x3 (보정)
  etcd 서버   ~/pgvip/vip_watch.sh                       crontab 매분, 20초 x3 (감시+알림)

확인
  ip -o -4 addr show | grep pgvip                       이 노드에 VIP 있나
  tail -f /pgdata/log/vip.log                           add/del 이력 (각 DB 노드)
  bash ~/pgvip/vip_watch.sh --once                      (etcd 서버) 리더 / VIP 위치
  tail ~/pgvip/vip_watch.log                            감시자 이력

수동 조치
  bash /postgres/app/etc/patroni/vip_check.sh --once    이 노드 즉시 보정
  sudo arping -U -c 3 -I <dev> <VIP>                    ARP 갱신 (접속 안 될 때)

vip_watch 알림 상태
  LEADER_CHANGED    WARN  failover/switchover 발생 -> 원인 확인 (patronictl history)
  VIP_FIXED         WARN  감시자가 보정함 -> callback 누락, 해당 노드 vip.log 확인
  VIP_FIX_FAILED    CRIT  보정하려는데 SSH 불가 -> 노드 상태 확인
  MULTI_LEADER      CRIT  리더 2개 -> 9-4 즉시 수행
  NO_LEADER         CRIT  리더 없음 -> 9-2 / 9-3
  NODE_UNREACHABLE  WARN  노드 SSH 불가
  VIP_NO_PING       WARN  위치는 맞는데 ping 안 됨 -> ARP/방화벽. ICMP 차단 환경이면
                          vip_watch.sh 상단 VIP_PING="n"
  정상으로 돌아오면 RESOLVED 발송. 상태는 2회 연속(약 20초) 같을 때만 발송

알림 채널 연결
  vip_watch.sh 상단 PGMON_HOME="<pgmon 경로>" 지정
  -> pgmon conf/notify.conf 의 채널(log / command / webhook) 로 발송
  비워두면 ~/pgvip/vip_watch.log 에만 기록

VIP 끄기 (임시)
  각 DB 노드 crontab 에서 vip_check 줄 주석, etcd 서버 crontab 에서 vip_watch 줄 주석
  patroni.yml 의 postgresql.callbacks 블록 삭제 -> patronictl -c $PYML reload $CL
  VIP 수동 제거: sudo ip addr del <VIP>/<prefix> dev <dev>
```

5. 안전장치 / 노드 추가 / DR (HA)

5-1. watchdog (펜싱)

```
역할  Patroni 가 멈추면 약 25초 뒤 커널(softdog)이 노드를 리부트
확인
  lsmod | grep softdog
  ls -l /dev/watchdog                       실행 계정 소유여야 함
  journalctl -u patroni | grep -i watchdog
  grep -A3 '^watchdog' $PYML                mode: automatic / required / off
주의
  - Patroni 를 kill -9 하면 watchdog 이 닫히지 않아 노드가 리부트될 수 있음
    정지는 항상 sudo systemctl stop patroni
  - 서버 리부트 후에도 softdog 은 /etc/modules-load.d/softdog.conf 로 자동 로드
  - 끄려면 양쪽 patroni.yml 의 watchdog.mode 를 "off" 로
    -> patronictl -c $PYML reload $CL
```

5-2. failsafe_mode

```
역할  etcd 전체 장애 시에도 Primary 가 모든 멤버와 REST(8008) 로 통신되면 유지
확인  patronictl -c $PYML show-config | grep failsafe
변경  patronictl -c $PYML edit-config   ->  failsafe_mode: true|false
전제  Primary <-> Standby 8008 포트 통신
```

5-3. synchronous_mode

```
역할  커밋을 Standby 에도 기록한 뒤 응답 -> failover 시 데이터 유실 0
대가  커밋 지연 증가 (네트워크 왕복 1회)
      Standby 장애 시 Patroni 가 자동으로 비동기 전환 -> 서비스는 계속
```

확인 patronictl list 의 Role 이 "Sync Standby"
변경 patronictl -c $PYML edit-config   -> synchronous_mode: true|false
※ synchronous_mode_strict 는 2노드에서 Standby 장애 시 쓰기가 멈추므로 비권장

5-4. 노드 추가 / DR 구성 - add_node.sh (HA)

실행 : etcd 서버의 패키지 디렉토리에서 bash add_node.sh
준비 : 새 서버에 사전 구성만 (계정/sudo, 최상위 디렉토리, /etc/hosts,
       방화벽, etcd 서버 -> 새 서버 SSH 키). PG 를 미리 설치하지 말 것

A) 같은 클러스터에 Standby 추가 - 3노드 HA 구성용
   기존 클러스터 멤버가 됨. failover 후보 여부 선택
     예   : Primary 장애 시 승격 대상, VIP 스크립트도 배포
     아니오 : tags.nofailover/nosync = true, 절대 Primary 안 됨
B) DR standby cluster - 다른 사이트 DR 구성용
   새 서버에 자기 etcd(1노드) + 별도 Patroni 클러스터(<클러스터>-dr)
   메인 VIP 를 따라 복제 (Standby Leader). 자동 승격 없음
   메인에 영구 복제 슬롯(<이름>_slot) 생성 -> DR 이 늦어도 WAL 유지

스크립트가 하는 일
  기존 노드 patroni.yml 읽기 -> 엔진 복사(etcd 서버 경유) -> rpm
  -> 기존 멤버 pg_hba 에 새 서버 허용 + reload -> patroni.yml/service
  -> (A) VIP 스크립트 / vip_watch 감시 대상 추가 (B) DR etcd 설치
  -> Patroni 설치/기동 -> 클론 완료(streaming)까지 대기

DR 전환 (메인 전체 장애, 사람이 판단 후)
  ssh <DR 서버>
  patronictl -c $PYML edit-config   -> standby_cluster 블록 삭제 후 저장
  = DR 이 Primary 로 승격. 앱 접속 주소를 DR 로 변경
  메인 복구 후에는 메인을 DR 을 따라가는 standby 로 재구성해야 함

DR 을 없앨 때
  메인에서 patronictl -c $PYML edit-config -> slots.<이름>_slot 삭제
  (슬롯이 남아 있으면 메인에 WAL 이 계속 쌓임)

주의
  - A 에서 failsafe_mode 사용 시: etcd 장애 중 Primary 가 모든 멤버와
    8008 로 통신돼야 Primary 유지 -> 새 노드와 8008 통신 필수
  - postgresql.conf 메모리 설정은 기존 노드 값을 그대로 씀
    (사양이 다르면 patronictl edit-config 로 조정)

6. 백업 / WAL 아카이브 / 복구

백업 방식은 설치 때 고른 것 (basebackup / pgbackrest). pgbackrest 는 6-7

스크립트 : /postgres/app/etc/backup/
  basebackup/pg_basebackup.sh   베이스 백업 (크론, 기본 매일 02:00)
  basebackup/backup_setup.sh   경로 / 주기 / 보관 개수 / 크론 재설정, 끄기
  wal_cleanup.sh   아카이브 WAL 정리 (크론 03:30 + 백업 직후)
  pitr_restore.sh   복구

HA : 백업 크론은 선택한 백업 서버에만, wal_cleanup 은 양쪽 노드

6-1. 수동 백업

bash /postgres/app/etc/backup/basebackup/pg_basebackup.sh
tail /pgdata/log/pg_basebackup.log
백업 디렉토리: /pgbackup/back/<YYYYMMDD_HHMMSS>/
               (base.tar.gz, pg_wal.tar.gz, WAL_START.txt)

6-2. 백업 설정 변경

bash /postgres/app/etc/backup/basebackup/backup_setup.sh

6-3. WAL 정리

bash /postgres/app/etc/backup/wal_cleanup.sh --dry-run   지울 대상만 출력
bash /postgres/app/etc/backup/wal_cleanup.sh

기준
  백업이 있는 서버 : 가장 오래된 보관 백업의 시작 WAL 이전만 삭제
  백업이 없는 서버 : KEEP_DAYS(보관 개수 x 주기 + 1일) 지난 WAL 삭제
  .history 파일은 지우지 않음

6-4. 아카이브 실패 확인

```
psql -c "select * from pg_stat_archiver;"
failed_count 증가 시: /pgbackup/arc 권한 / 디스크 여유, PG 로그의 archive 에러
```

6-5. 복구 - single

```
bash pitr_restore.sh --list                                  백업 목록
bash pitr_restore.sh                                         최신 백업 + WAL 끝까지
bash pitr_restore.sh --target-time "2026-11-08 14:30:00"     그 시각 직전까지
bash pitr_restore.sh --backup 20261108_020000 --yes          특정 백업, 확인 생략
동작
  PG 정지 -> 현재 DATA 는 /pgdata/data.before_restore_<시각> 으로 보존
  -> 백업 풀기 -> restore_command + recovery.signal -> 기동 -> 복구 완료 대기
복구 확인 후 보존본 삭제 (디스크 확인)
```

6-6. 복구 - HA

```
1) patronictl -c $PYML pause
2) 양쪽 노드: sudo systemctl stop patroni
3) 복구할 노드(보통 기존 Primary): bash pitr_restore.sh [--target-time ...]
   ※ 백업은 백업 서버에 있으므로 필요하면 같은 경로(/pgbackup/back)로 먼저 복사
4) 복구 확인 -> /postgres/app/bin/pg_ctl -D /pgdata/data stop -m fast
   -> sudo systemctl start patroni -> patronictl -c $PYML resume
5) patronictl -c $PYML reinit $CL <다른 노드>
```

6-7. pgBackRest (백업 선택 = pgbackrest)

```
위치 (모든 서버)  /postgres/app/etc/backup/pgbackrest/
                  bin/pgbackrest  conf/pgbackrest.conf  log/  pgbr_restore.sh
저장소            single: 이 서버 /pgbackup/pgbackrest
                  HA    : etcd 서버 /pgbackup/pgbackrest
stanza            single: main / HA: 클러스터 이름 (pg-cluster)
크론              저장소 서버 (HA 는 etcd 서버)
                  일요일 02시 full + 월~토 02시 diff, full 2개 보관
                  오래된 백업 / WAL 은 백업 끝에 pgBackRest 가 자동 정리 (expire)

상태 / 목록       pgbackrest --stanza=pg-cluster info
점검              pgbackrest --stanza=pg-cluster check     (WAL 아카이브까지 확인)
수동 백업         pgbackrest --stanza=pg-cluster --type=full backup   (저장소 서버에서)
                  --type=diff : 마지막 full 이후 변경분
                  --type=incr : 마지막 백업 이후 변경분
로그              /postgres/app/etc/backup/pgbackrest/log/
                    <stanza>-backup.log, <stanza>-archive-push.log (DB 노드),
                    cron_<stanza>.log
아카이브 실패     psql -c "select * from pg_stat_archiver;"  failed_count 증가 시
                  DB 노드 log/<stanza>-archive-push.log 확인
                  HA: DB 노드 -> etcd 서버 SSH 확인
                    ssh -o BatchMode=yes <etcd서버> true
                  ※ etcd 서버가 내려가면 archive-push 가 실패하고 WAL 이
                    pg_wal 에 쌓인다 (서비스는 계속). etcd 서버 복구 후 자동 해소.
                    오래 걸리면 pg_wal 디스크 확인
                  ※ Standby 가 내려가 있으면 check 는 실패하지만
                    백업은 Primary 에서 진행 (backup-standby=prefer)

복구 - single     bash /postgres/app/etc/backup/pgbackrest/pgbr_restore.sh --list
                  bash .../pgbr_restore.sh                                  최신까지
                  bash .../pgbr_restore.sh --target-time "2026-11-09 14:30:00"
                  bash .../pgbr_restore.sh --set 20261109-020001F           특정 백업
                  bash .../pgbr_restore.sh ... --delta    DATA 보존 없이 덮어씀 (빠름)
                  복구 후 새 타임라인 -> 바로 full 백업 1회
복구 - HA         1) patronictl -c $PYML pause
                  2) 양쪽 노드: sudo systemctl stop patroni
                  3) 기존 Primary 노드: bash .../pgbr_restore.sh [--target-time ...] --yes
```

```
   (저장소가 etcd 서버라 백업 복사 불필요. 끝나면 PG 를 다시 내려둠)
4) 그 노드: sudo systemctl start patroni -> patronictl ... resume
5) 다른 노드: sudo systemctl start patroni
   -> patronictl -c $PYML reinit $CL <다른 노드>
6) etcd 서버: pgbackrest --stanza=$CL --type=full backup
※ HA 복구 절차는 실서버 미검증. 운영 전 리허설 필수

노드 추가        add_node.sh (A: 3-node) 가 새 노드를 저장소 설정에 pg3-host 로 추가
                DR (B) 는 별도 클러스터라 추가 안 함 (restore_command 로 메인 저장소 사용)
```

7. pgmon 모니터링

```
실행 : bash <pgmon 경로>/pgmon.sh
화면 : 1 대시보드 / 2 세션 / 3 Statement / 4 Vacuum / 5 Object / 6 Alert
       7 설정 / 8 Report / 9 복제
```

7-1. 수집 데몬 (root 불필요)

```
bash pgmon_ctl.sh status
bash pgmon_ctl.sh start   [닉네임|all]
bash pgmon_ctl.sh stop    [닉네임|all]       watchdog 재기동 대상에서도 빠짐
bash pgmon_ctl.sh restart [닉네임|all]
bash pgmon_ctl.sh cron-install               재부팅 자동 시작 + 5분 watchdog
bash pgmon_ctl.sh log <닉네임> [줄수]
bash pgmon_check.sh                          설치 / 수집 점검 리포트
bash pgmon_check.sh --alert-test             Alert 발생 / 해소까지 시험
```

7-2. 알림 설정

```
cp conf/notify.conf.sample conf/notify.conf
NOTIFY_CHANNELS="log,command"       log / command / webhook
NOTIFY_MIN_SEVERITY="WARN"          WARN | CRIT
NOTIFY_COMMAND='... 메일 / 메신저 스크립트 ...'
  전달 변수: PGMON_EVENT PGMON_SEVERITY PGMON_DB PGMON_METRIC
             PGMON_VALUE PGMON_MESSAGE
bash pgmon_ctl.sh restart
알림 이력: conf/alerts.log
```

7-3. 임계값

```
6) Alert 화면 -> 임계값 (전역 / DB별)
대상: CPU / MEM / SWAP / Lock / idle in tx / long session / 복제 지연(초) /
      슬롯 WAL 적체(MB) / 테이블 dead ratio(%)
```

7-4. 보관 / 주기

```
7) 설정 화면 -> 메트릭별 주기, ON/OFF, 보관 기간(기본 15일)
```

7-5. OS 지표가 "수집 불가"일 때

```
DB 방식: GRANT pg_read_server_files TO pgmon_monitor;
         GRANT EXECUTE ON FUNCTION pg_read_file(text) TO pgmon_monitor;
SSH 방식: pgmon 서버 -> DB 서버 키 교환 (등록 시 입력한 SSH 유저)
```

8. 쿼리 튜닝 - pg_tune

```
pgmon 에서 : 3) Statement -> t -> 번호 선택
  - 파라미터($1, $2) 가 있으면 실제 값을 넣은 SQL 입력
  - 실행 계정은 업무 테이블 조회 권한이 있는 계정
  - 기본 EXPLAIN 만, EXPLAIN ANALYZE 는 확인 후 실행
  - 결과 로그: pgmon/conf/pgtune_log/

CLI 로 (postgresql/tools/pg_tune)
  T=postgresql/tools/pg_tune/pgtune.py
  python3 $T tune        --dsn "$DSN" --sql "SELECT ..."            추정 플랜
  python3 $T tune        --dsn "$DSN" --sql "SELECT ..." --analyze  실측 (DML 은 ROLLBACK)
  python3 $T verify      --dsn "$DSN" --sql "..." --index "CREATE INDEX ON t (c)"
  python3 $T hint        --dsn "$DSN" --sql "..." --hint "HashJoin(o c)"
  python3 $T stats       --dsn "$DSN" --sql "..."
```

```
  python3 $T index-check --dsn "$DSN" --min-scans 50

tune 순서
  진단(규칙 15종) -> 테이블 통계 -> 인덱스 후보(hypopg) ->
  실행계획 대안(pg_hint_plan: Hash/Merge Join, 인덱스 강제, Rows 보정) ->
  설정 실험(work_mem, jit, --analyze 일 때) -> 추천
  추천 우선순위: 인덱스 > 쿼리 수정 > 통계 > 설정 > 힌트(최후 수단)

준비
  hypopg       대상 DB 에 CREATE EXTENSION hypopg;
  pg_hint_plan 설치만 돼 있으면 됨, superuser 계정으로 실행해야 실험함
  둘 다 없으면 해당 단계만 "검증 불가" 로 나오고 나머지는 동작
```

9. 장애 대응 플레이북

```
공통 첫 단계 (HA)
  patronictl -c $PYML list
  bash ~/pgvip/vip_watch.sh --once           (etcd 서버)
  journalctl -u patroni -n 200 --no-pager    (해당 노드)
```

9-1. Primary 장애 -> 자동 failover 됨

```
  확인  patronictl list : 새 Leader, 구 Primary 는 없음 또는 stopped
        vip_watch : OK, 리더 = 새 노드
  조치  구 Primary 서버 복구 -> sudo systemctl start patroni
        -> 자동으로 Replica 재합류 (pg_rewind). 실패 시 reinit (3-6)
  선택  원래 노드로 되돌리기: 업무 영향 적은 시간에 switchover (3-3)
```

9-2. failover 가 안 됨 (NO_LEADER 지속)

```
  원인 후보
    - pause 상태     -> patronictl list 상단 "Maintenance mode" -> resume
    - etcd 장애      -> etcdctl --endpoints=<etcd들>:2379 endpoint health
    - Standby 지연이 maximum_lag_on_failover(1MB) 초과 -> 승격 거부
      긴급 시 patronictl -c $PYML failover $CL --candidate <standby> (유실 감수)
    - Standby Patroni 정지 -> sudo systemctl start patroni
```

9-3. etcd 장애

```
  3노드 1대 장애 : 영향 없음. 해당 노드 sudo systemctl start etcd
  전체 / 1노드 구성 장애 :
    failsafe_mode=true  -> Primary 유지 (failover 불가 상태)
    failsafe_mode=false -> Primary demote, 쓰기 중단
    etcd 복구가 최우선: sudo systemctl status etcd, journalctl -u etcd
    etcd 데이터 손상 시: 멤버 재구성 필요 (/var/lib/etcd)
```

9-4. split-brain 의심 (MULTI_LEADER)

```
  1) 양쪽 노드 확인
       psql -Atc "select pg_is_in_recovery();"     f 가 둘이면 진짜 P-P
       curl -s -o /dev/null -w '%{http_code}' http://<노드>:8008/primary
  2) patronictl list 에서 진짜 Leader 확인 (etcd 기준)
  3) Leader 가 아닌 쪽 즉시 정지
       sudo systemctl stop patroni
       (안 되면) /postgres/app/bin/pg_ctl -D /pgdata/data stop -m fast
  4) VIP 가 Leader 에만 있는지 확인 (4장)
  5) 정지한 쪽에 들어간 쓰기가 있었는지 업무 확인 (데이터 대조)
  6) 정지한 노드 reinit 후 재합류
  7) 원인 분석: watchdog 상태, 네트워크, 시간 동기화
```

9-5. 복제 지연 / 끊김

```
  psql -c "select * from pg_stat_replication;"            (Primary)
  psql -c "select * from pg_stat_wal_receiver;"           (Standby)
  pgmon 9) 복제 화면 : 지연 bytes / 초, 슬롯 적체
  네트워크 / Standby 디스크 IO / 긴 쿼리(hot_standby 충돌) 확인
  복구 불가 시 reinit (3-6)
```

9-6. 아카이브 디스크 풀

```
df -h /pgbackup
bash wal_cleanup.sh --dry-run -> 정리 기준 확인
백업이 오래 실패해서 정리가 안 되는 경우가 많음 -> 백업 먼저 성공시키고 정리
※ pg_wal 디렉토리 파일을 직접 지우지 말 것
```

9-7. 노드가 갑자기 리부트됨

```
watchdog 동작 가능성 : journalctl -b -1 | grep -i -E 'watchdog|softdog'
직전 Patroni 상태 : journalctl -u patroni -b -1 | tail -50
원인 : Patroni hang, 메모리 부족(OOM), IO 멈춤 -> 해결 후 재합류 확인
```

9-8. VIP 로 접속 안 됨

```
vip_watch --once 로 위치 확인 -> 리더 노드에 VIP 있는지
있으면 : sudo arping -U -c 3 -I <dev> <VIP>, 앱 서버 arp -n | grep <VIP>
없으면 : bash vip_check.sh --once, /pgdata/log/vip.log, sudo -n ip 동작 확인
```

10. 로그 위치 모음

PostgreSQL	/pgdata/log/postgresql-YYYY-MM-DD.log
설치	/pgdata/log/pg_install.log, ha_setup.log
Patroni	journalctl -u patroni
etcd	journalctl -u etcd
VIP (노드)	/pgdata/log/vip.log
VIP 감시 (etcd 서버)	~/pgvip/vip_watch.log
백업	/pgdata/log/pg_basebackup.log
pgBackRest	/postgres/app/etc/backup/pgbackrest/log/ (HA 저장소: etcd 서버)
WAL 정리	/pgdata/log/wal_cleanup.log
pgmon collector	pgmon/conf/collector_<닉네임>.log
pgmon 알림	pgmon/conf/alerts.log
pgmon watchdog	pgmon/conf/watchdog.log
pg_tune	pgmon/conf/pgtune_log/

11. 정기 작업 / 변경 작업

11-1. OS 패치 / 재부팅 (HA, 무중단에 가깝게)

```
1) Replica 노드: sudo systemctl stop patroni -> 패치 -> 재부팅
   -> patronictl list 로 streaming 복귀 확인
2) patronictl -c $PYML switchover $CL     (패치 끝난 노드를 Leader 로)
3) 기존 Primary 노드 동일하게 패치
※ 작업 중 다른 노드가 죽을 위험이 걱정되면 pause 대신 위 순서 유지
  (pause 상태에서는 장애 시 failover 가 안 됨)
```

11-2. PG 마이너 업그레이드 (수동)

```
다운로드 서버: BUILD_LOCK.txt 의 PG 버전만 새 마이너로 바꿔서
              pg_download.sh --lock BUILD_LOCK.txt
폐쇄망에서 같은 configure 옵션으로 새 경로에 빌드
  (옵션: /postgres/app/etc/configure_options.txt)
single : 정지 -> 엔진 교체 -> 기동
HA     : Replica 먼저 교체 -> switchover -> 나머지 교체
extension 도 새 엔진 기준으로 다시 빌드 (PGXS)
```

11-3. 파라미터 튜닝

```
single : postgresql.conf 수정 -> pgreload (재시작 필요 항목은 pgstop/pgstart)
HA     : patronictl edit-config -> reload / restart (3-2)
```

11-4. 패스워드 변경

```
psql -c "ALTER ROLE postgres PASSWORD '...';"
HA : patroni.yml 의 authentication.superuser / replication password 도 같이 변경
     -> patronictl -c $PYML reload $CL
     복제 유저를 바꾸면 ~/.pgpass 도 갱신
```

END